SoniType: A Perceptual Type System and Optimizing Compiler for Information-Preserving Data Sonification

SoniType Contributors
Independent Research
USA
sonitype@example.org

Abstract

Data sonification—the systematic mapping of data variables to auditory parameters—remains a craft practice despite five decades of research. Practitioners hand-tune pitch ranges, timbral mappings, and temporal envelopes by trial and error, with no formal guarantee that the resulting audio conveys the intended information. We present SoniType, a domain-specific language and optimizing compiler that treats sonification as a *lossy coding problem* over a psychoacoustically-constrained perceptual channel. SoniType introduces a *perceptual type system* in which psychoacoustic constraints—critical-band masking thresholds, Weber-fraction just-noticeable differences (JNDs), Bregman stream segregation predicates, and cognitive load budgets—serve as first-class type qualifiers. A specification that type-checks is guaranteed, relative to the psychoacoustic model, to produce perceptually discriminable audio. The compiler maximizes *psychoacoustically-constrained mutual information* $I_\psi(D;A)$ via branch-and-bound search with constraint propagation, then generates bounded-latency renderers with static worst-case execution time (WCET) guarantees. Our implementation comprises 70,035 lines of Rust across 11 workspace crates. Automated model-based evaluation on four real-world datasets suggests that SoniType can substantially improve model-predicted discriminability and constraint satisfaction relative to simple algorithmic baselines, though the gains are not uniform across all proxy metrics and have not been validated with human listeners. Compilation completes in under 5 ms for datasets up to 500 data points (mean <1 ms).

CCS Concepts

• **Software and its engineering** → **Domain specific languages**;
• **Human-centered computing** → **Accessibility technologies**.

Keywords

sonification, perceptual type system, psychoacoustics, domain-specific language, compiler, accessibility, auditory display

1 Introduction

Data sonification—the systematic mapping of data variables to auditory parameters such as pitch, loudness, timbre, and spatial position—offers a complementary channel to visualization for conveying complex, temporal, and multivariate information [1, 2]. Despite five decades of research, sonification design remains a craft practice: practitioners hand-tune parameter ranges and timbral assignments through iterative listening, with no formal guarantee that the resulting audio stream preserves the intended data relationships [3].

The core dysfunction is that sonification design lacks two properties that enabled visualization to mature as a discipline: (1) a declarative specification language with composable semantics, and (2) an optimizing backend that automatically negotiates the constraints of the output channel. We observe that the human auditory system is, formally, a *noisy channel* with well-characterized capacity limits—critical-band masking, Weber-fraction just-noticeable differences (JNDs), and Bregman-style stream segregation constraints [4, 5]—and that sonification is therefore a *lossy coding problem* over a psychoacoustically-constrained perceptual channel.

The practical consequences of this gap are severe. Medical alarm fatigue is a documented patient-safety crisis: the ECRI Institute has listed alarm hazards as the #1 health technology hazard in multiple years [6], and the Joint Commission issued a National Patient Safety Goal for clinical alarm management [7]. A typical ICU has 30+ distinct alarm types, creating 435+ pairwise interactions that must be perceptually discriminable. The IEC 60601-1-8 standard prescribes that "alarms should be distinguishable" [8], but provides no computational tool to verify this property.

We present SoniType, a domain-specific language (DSL) and optimizing compiler that closes this loop. SoniType accepts declarative data-to-sound mapping specifications and compiles them into bounded-latency audio renderers while maximizing the *perceptual mutual information* between input data and perceived audio. The key intellectual contribution is a *perceptual type system* in which psychoacoustic constraints—masking thresholds, JND bounds, stream segregation predicates, and cognitive load budgets—are first-class type qualifiers. A specification that type-checks is guaranteed (relative to the psychoacoustic model) to produce perceptually discriminable, non-masked, cognitively tractable audio.

Contributions.

(1) A **perceptual type system** where refinement predicates derive from psychophysics, with novel challenges from non-local cross-stream interactions and non-convex feasibility regions (§4).
(2) **Psychoacoustically-constrained mutual information** $I_\psi(D;A)$ as an optimization objective, with a $(1-1/e)$-approximation algorithm exploiting Bark-band decomposition (§6).
(3) **Decidable stream segregation predicates**: the first computationally tractable formalization of Bregman's auditory scene analysis principles (§5.3).
(4) **Bounded-latency code generation** with static WCET analysis for real-time audio callbacks (§7).
(5) A **perceptual linter** for existing sonification and alarm designs (§8).

(6) An open-source implementation comprising **70,035 lines of Rust** across 11 workspace crates (§10).

2 Related Work

2.1 Sonification Frameworks and Tools

The sonification community has produced several tools that make auditory display accessible to non-specialists. The *Sonification Sandbox* [9] from Georgia Tech provides a GUI-based environment for mapping data to pitch, loudness, and timbre, but offers no formal constraint checking. NASA's *xSonify*[1] targets space physics data with accessible interfaces but lacks perceptual optimization. *TwoTone* [10] offers browser-based sonification with presets for common data types, prioritizing accessibility over perceptual rigor. The *Highcharts Sonification Module* [11] provides JavaScript-based sonification for web dashboards with some accessibility features but no psychoacoustic verification.

In the scientific domain, *Astronify*[2], developed at the Space Telescope Science Institute, provides sonification of astronomical time-series data from missions like JWST and Kepler. While Astronify excels at single-stream pitch mappings with accessibility focus, it does not handle multi-stream compositions or provide formal masking/segregation verification. The *matplotlib sonification plugins* (`audplot`) extend Python's dominant visualization library to audio, providing convenient integration with data science workflows but inheriting matplotlib's non-declarative API and offering no psychoacoustic optimization.

Table 1 provides a systematic comparison of existing sonification tools against SoniType across key capability dimensions.

2.2 Audio Programming Languages

SuperCollider [12] provides a powerful object-oriented language for audio synthesis and algorithmic composition. *Faust* [13] is a functional DSL for DSP that compiles to efficient C++ with formal signal-flow semantics. *Csound* [14] offers decades of unit generators and orchestral modeling. *Pure Data* (Pd) [47] provides a visual dataflow environment widely used in sound art and interactive music; its graphical patching paradigm offers rapid prototyping but no formal perceptual verification. *ChucK* [15] introduces strongly-timed programming for audio. These systems are *audio compilers*: they compile audio synthesis programs efficiently. They are agnostic to data semantics and perceptual constraints—they do not know that a synthesized signal is meant to *represent data*, nor can they verify that the representation is perceptually effective.

2.3 Psychoacoustic Models in Computing

Psychoacoustic models are widely used in perceptual audio coding (MP3, AAC, Opus) [16, 17] to *discard* information below perceptual thresholds. SoniType inverts this paradigm: we use the same class of models to *maximize* information transmission through the auditory channel. The Zwicker/Schroeder masking model [5, 18] and the Glasberg-Moore loudness model [19] provide the perceptual cost functions we integrate into the compiler.

[1]https://sourceforge.net/projects/xsonify/
[2]https://github.com/spacetelescope/astronify

2.4 Refinement Type Systems

Refinement types [20, 21] attach logical predicates to types, enabling richer static verification. *Liquid Types* [21] combine Hindley-Milner inference with SMT-backed refinement checking. SoniType's perceptual type system extends this paradigm: refinement predicates derive from psychophysics rather than program logic, and the non-local nature of masking interactions creates challenges absent from standard refinement type frameworks.

2.5 Domain-Specific Compilers

SoniType's architecture follows the separation-of-concerns principle pioneered by *Halide* [24] for image processing: where Halide separates algorithm from schedule over a hardware target, SoniType separates data semantics from perceptual rendering over a human auditory target. *Spiral* [25] applies similar principles to DSP, auto-tuning implementations for specific hardware. The key difference is that SoniType's "hardware target" is the human auditory system, with psychoacoustic constraints replacing architectural constraints.

2.6 Accessibility and Auditory Display

The sonification research community has established guidelines for accessible auditory display [1, 3]. The Web Content Accessibility Guidelines (WCAG 2.2) [26] and the EU Accessibility Act [27] drive increasing demand for non-visual data representations. Flowers et al. [28] document the effectiveness of sonification for data comprehension by blind users. SoniType's accessibility module directly addresses these needs with hearing profile adaptation, frequency remapping, and cognitive support features.

3 The SoniType DSL

The SoniType DSL provides a declarative syntax for specifying data-to-sound mappings. A complete specification consists of *data declarations* (schema, range, semantics), *stream definitions* (mapping expressions with optional constraint annotations), and *composition blocks* that trigger perceptual verification.

```
data heart_rate: TimeSeries<Float> {
  range: 40.0..200.0, unit: "bpm"
}
data blood_pressure: TimeSeries<Float> {
  range: 60.0..220.0, unit: "mmHg"
}
data spo2: TimeSeries<Float> {
  range: 70.0..100.0, unit: "percent"
}

stream hr = map heart_rate {
  pitch: log(80Hz..500Hz),
  timbre: sine, pan: left(40deg)
}
stream bp = map blood_pressure {
  pitch: linear(400Hz..1200Hz),
  timbre: saw, pan: right(40deg)
}
stream o2 = map spo2 {
  pitch: linear(1000Hz..3000Hz),
  timbre: triangle, pan: center
```

Table 1: Systematic comparison of sonification tools across capability dimensions. ✓= full support, ∼ = partial support, — = not supported.

Tool	Year	Multi-stream	Masking check	JND verify	Segregation	Cog. load	Optimization	Real-time	Accessibility
Son. Sandbox [9]	2003	∼	N/A	N/A	N/A	N/A	N/A	N/A	∼
xSonify	2006	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✓
TwoTone [10]	2020	∼	N/A	N/A	N/A	N/A	N/A	∼	∼
Highcharts [11]	2022	∼	N/A	N/A	N/A	N/A	N/A	✓	✓
Astronify	2023	N/A	N/A	N/A	N/A	N/A	N/A	N/A	✓
SuperCollider [12]	2002	✓	N/A	N/A	N/A	N/A	N/A	✓	N/A
Faust [13]	2009	✓	N/A	N/A	N/A	N/A	∼	✓	N/A
Csound [14]	2000	✓	N/A	N/A	N/A	N/A	N/A	✓	N/A
Pure Data [47]	1996	✓	N/A	N/A	N/A	N/A	N/A	✓	N/A
SoniType	**2025**	✓	✓	✓	✓	✓	✓	✓	✓

```
}

compose icu_monitor {
  streams: [hr, bp, o2],
  sample_rate: 44100, buffer_size: 256
}
```

Listing 1: Three-stream ICU monitor specification.

The parser (`parser.rs`, 1,170 lines) implements a recursive-descent grammar producing a typed AST. The lexer (`lexer.rs`, 659 lines) recognizes domain-specific literals: frequency (`440Hz`), perceptual units (`3Bark`, `500mel`), angles (`30deg`), and durations (`100ms`).

3.1 DSL Design Principles

The SoniType DSL is designed around four principles:

Declarative data semantics. Users specify *what* data to sonify and the intended mapping semantics (e.g., "price maps to pitch logarithmically"), not *how* the audio synthesis should be implemented. This separation—analogous to Halide's algorithm/schedule separation [24]—enables the compiler to freely optimize the rendering without changing the data semantics.

Perceptual constraints as first-class citizens. Users can annotate streams with perceptual requirements (e.g., `jnd_margin: >= 2.0`) that become compile-time contracts. The type checker rejects specifications that violate these constraints, providing early feedback before any audio is generated.

Composability. Streams are independently defined and composed via `compose` blocks. Perceptual constraints are checked at composition boundaries—adding a stream to a composition triggers pairwise masking and segregation verification against all existing streams.

Progressive disclosure. Simple specifications (single-stream, default constraints) require minimal syntax. Advanced features (custom hearing profiles, explicit segregation requirements, WCET budgets) are opt-in extensions that do not burden novice users.

3.2 Example: Multivariate Stock Data

The following specification demonstrates a realistic financial data sonification:

```
data close: TimeSeries<Float> {
  range: 100.0..5000.0, unit: "USD"
}
data volume: TimeSeries<Float> {
  range: 0.0..1e10, unit: "shares"
}
data volatility: TimeSeries<Float> {
  range: 0.0..0.1, unit: "ratio"
}

stream price = map close {
  pitch: log(220Hz..1760Hz),
  timbre: sine { harmonics: [1.0, 0.3, 0.1] }
}
stream vol = map volume {
  loudness: power(45dB..75dB, 0.5),
  carrier: price
}
stream vol_indicator = map volatility {
  timbre_roughness: linear(0.0..0.8),
  carrier: price
}

compose stock_sonification {
  streams: [price, vol, vol_indicator],
  sample_rate: 44100, buffer_size: 512
}
where {
  pitch_jnd_margin >= 2.0,
  masking_clearance >= 6dB,
  cognitive_load <= 3
}
```

Listing 2: Stock price sonification with volume and volatility.

This 30-line specification replaces a substantially longer SuperCollider implementation, including the psychoacoustic verification that would require separate manual analysis.

3.3 Data Input Formats

The SoniType DSL supports two primary data input mechanisms:

CSV (Comma-Separated Values). CSV is the most common tabular data format in data science workflows. SoniType's CSV reader (`sonitype-streaming/src/csv_input.rs`) provides:

- Automatic column type detection (numeric vs. categorical vs. temporal)
- Missing value handling (interpolation, forward-fill, skip, or default)
- Statistical summary computation (min, max, mean, standard deviation) for optimizer hints
- Streaming mode for large files (row-by-row iteration without full memory loading)

HDF5 (Hierarchical Data Format). HDF5 is standard in scientific computing for multi-dimensional array data (climate models, telescope observations, simulations). SoniType reads HDF5 metadata via JSON sidecar files to avoid platform-specific C library dependencies, with optional native HDF5 support via feature flags. The HDF5 reader supports extraction of 1D time-series slices from multi-dimensional datasets for sonification.

3.4 Output Formats

SoniType generates two audio output formats:

WAV (Waveform Audio File Format). PCM WAV files at 16-bit, 24-bit integer, or 32-bit float precision. The WAV writer supports optional peak normalization (−0.1 dBFS) and dithering for bit-depth reduction. WAV is the primary output format for offline rendering and file-based sonification delivery.

MIDI (Musical Instrument Digital Interface). Standard MIDI File (SMF) export maps perceptual parameters to MIDI events: pitch frequency → note number (via $69 + 12\log_2(f/440)$), loudness → velocity, timbre → program change, pan → CC#10. MIDI output enables integration with external synthesizers, digital audio workstations, and custom rendering pipelines for users requiring timbral capabilities beyond SoniType's built-in oscillators.

4 Perceptual Type System

4.1 Perceptual Refinement Types

SoniType's type system augments standard Hindley-Milner inference [22, 23] with *perceptual refinement qualifiers*. Each stream type carries predicates drawn from psychoacoustics:

$$\tau ::= \tau_{\text{base}} \mid \{v : \tau_{\text{base}} \mid \phi_{\text{mask}}(v, \Gamma) \wedge \phi_{\text{jnd}}(v) \wedge \phi_{\text{seg}}(v, \Gamma) \wedge \phi_{\text{cog}}(\Gamma)\} \tag{1}$$

where Γ is the typing environment containing all concurrently active streams, and:

- $\phi_{\text{mask}}(v, \Gamma)$: masking clearance exceeds threshold in all 24 Bark bands
- $\phi_{\text{jnd}}(v)$: parameter differences exceed JNDs in all dimensions
- $\phi_{\text{seg}}(v, \Gamma)$: at least one Bregman segregation predicate holds for each stream pair
- $\phi_{\text{cog}}(\Gamma)$: total cognitive load within budget

4.2 Non-Local Constraint Checking

The critical novelty is that ϕ_{mask} and ϕ_{seg} depend on Γ: adding stream S_{k+1} to a well-typed composition $\{S_1, \ldots, S_k\}$ requires rechecking $O(k)$ pairwise constraints. This non-locality is inherent to the domain—spectral masking is a physical interaction between concurrent sounds—and distinguishes SoniType from standard refinement type systems where predicates are properties of individual terms.

$$\frac{\Gamma \vdash e_i : \text{Stream}_i \quad \forall (i,j) : \phi_{\text{mask}}(S_i, S_j) \wedge \phi_{\text{seg}}(S_i, S_j) \quad \phi_{\text{cog}}(\{S_1, \ldots, S_k\})}{\Gamma \vdash \text{compose}(e_1, \ldots, e_k) : \text{MultiStream}} \tag{2}$$

4.3 Executable Type-Checking Engine

We provide a reference implementation of the perceptual type checker in `perceptual_type_checker.py`, mirroring the Rust `JndValidator` logic. Each audio dimension carries a *perceptual type*—Pitch(f_{lo}, f_{hi}, *scale*), Duration($t_{\min}$, $t_{\max}$), Timbre(c_{lo}, c_{hi}), Loudness(ℓ_{lo}, ℓ_{hi})—whose range is validated against the corresponding Weber-fraction JND model. The checker counts JND-separated steps n_{steps} in each range and verifies $n_{\text{steps}} \geq 1$ (a range below one JND conveys no information). For multi-stream compositions, it computes the pairwise compound d' (Eq. 5.4) and rejects any pair falling below the configurable threshold ($\tau_d = 1.5$ by default, $\approx 93\%$ correct 2AFC). A `PerceptualLossEstimator` then quantifies the information loss per mapping as $\Delta I = \max(0, \log_2 |\mathcal{D}| - \log_2 n_{\text{steps}})$ bits, giving practitioners an immediate measure of channel utilisation.

5 Psychoacoustic Cost Models

5.1 Critical-Band Masking

We implement the Zwicker/Schroeder model [5, 18] of simultaneous masking across 24 Bark bands. The spreading function models energy propagation:

$$S(\Delta z) = 15.81 + 7.5(\Delta z + 0.474) - 17.5\sqrt{1 + (\Delta z + 0.474)^2} \quad [\text{dB}] \tag{3}$$

where Δz is the distance in Bark between masker and maskee. The masking clearance for stream i at Bark band b is:

$$M_{\text{clear}}(i, b) = E_i(b) - T_{\text{mask}}(b) - \sum_{j \neq i} S(z_j - z_b) \cdot E_j(b) \tag{4}$$

where $E_i(b)$ is the spectral energy of stream i in band b and $T_{\text{mask}}(b)$ is the masked threshold.

5.2 JND Validation

We validate parameter differences against Weber-fraction JND models across five perceptual dimensions:

$$\text{JND}_{\text{pitch}}(f) = \begin{cases} 0.003f & f < 500\,\text{Hz} \\ 0.006f & f \geq 500\,\text{Hz} \end{cases} \tag{5}$$

The compound JND across dimensions follows a Euclidean combination under perceptual independence:

$$d'_{\text{compound}} = \sqrt{\sum_d \left(\frac{\Delta p_d}{\text{JND}_d}\right)^2} \tag{6}$$

5.3 Stream Segregation Predicates

We formalize Bregman's grouping principles [4] as decidable Boolean predicates:

$$\phi_{\text{onset}}(S_i, S_j) \equiv |\Delta t_{\text{onset}}| > \tau_{\text{sync}} \tag{7}$$

$$\phi_{\text{harm}}(S_i, S_j) \equiv \neg(\text{harmonicity}(S_i, S_j) > \epsilon_h) \tag{8}$$

$$\phi_{\text{spec}}(S_i, S_j) \equiv |\Delta f_{\text{centroid}}|_{\text{Bark}} > \delta_{\text{Bark}} \tag{9}$$

$$\phi_{\text{fate}}(S_i, S_j) \equiv |\Delta \text{AM}_{\text{rate}}| > \rho \vee |\Delta \text{FM}_{\text{rate}}| > \rho \tag{10}$$

Streams S_i and S_j segregate if at least one predicate holds. For k streams, the segregation matrix requires $O(k^2)$ pairwise evaluations, each computable in constant time.

5.4 Compound Discriminability Index

Following signal detection theory [34], we compute a compound discriminability index d'_{compound} that integrates JND margins across all perceptual dimensions:

$$d'_{\text{compound}}(S_i, S_j) = \sqrt{\sum_{d \in \mathcal{D}} \left(\frac{\Delta_d(S_i, S_j)}{\sigma_d}\right)^2} \tag{11}$$

where $\mathcal{D} = \{\text{pitch, loudness, timbre, spatial, temporal}\}$ is the set of perceptual dimensions, Δ_d is the difference in dimension d, and σ_d is the JND in that dimension. The compound d' assumes perceptual independence across dimensions—a simplification, but one well-supported by psychoacoustic evidence for the dimensions used [29].

The typing rule for stream composition requires $d'_{\text{compound}} \geq \tau_d$ for all stream pairs, where τ_d is a user-configurable threshold (default 1.5, corresponding to approximately 93% correct discrimination in a 2AFC task).

5.5 Subtyping and Qualifier Lattice

Perceptual qualifiers form a lattice ordered by constraint strictness:

$$q_1 \preceq q_2 \iff \forall \phi \in q_2 : \phi \in q_1 \tag{12}$$

A stream with stricter constraints (larger JND margins, more masking clearance) is a subtype of one with looser constraints. This enables qualifier polymorphism: a function expecting a stream with ≥ 6 dB masking clearance accepts any stream with ≥ 8 dB.

$$\frac{\Gamma \vdash e : \{v : \tau \mid q_1\} \quad q_1 \preceq q_2}{\Gamma \vdash e : \{v : \tau \mid q_2\}} \quad \text{(T-Sub)} \tag{13}$$

5.6 Cognitive Load Model

Following Cowan's model [30] of working memory capacity:

$$\phi_{\text{cog}}(\{S_1, \ldots, S_k\}) \equiv \sum_{i=1}^{k} w(S_i) \leq C_{\max} \tag{14}$$

where $w(S_i)$ is the attentional weight of stream i (based on complexity, familiarity, and temporal density) and $C_{\max} \approx 4 \pm 1$.

The attentional weight function $w(S_i)$ is computed as:

$$w(S_i) = w_{\text{base}} \cdot (1 + \alpha_{\text{complex}} \cdot C(S_i)) \cdot (1 - \beta_{\text{famil}} \cdot F(S_i)) \tag{15}$$

where $C(S_i)$ is the information rate of stream i (bits per second), $F(S_i) \in [0, 1]$ is a familiarity coefficient (higher for common mappings like "pitch = value"), and $\alpha_{\text{complex}}, \beta_{\text{famil}}$ are empirically calibrated weighting parameters. The base weight $w_{\text{base}} = 1.0$ for a simple sine-tone pitch mapping; complex timbral streams or rhythmic patterns receive higher weights.

This model enables SoniType to reject specifications where streams are individually well-formed and pairwise discriminable, but collectively exceed the listener's cognitive capacity. For example, a 6-stream specification where each stream maps a different variable to a different auditory parameter might pass pairwise masking and segregation checks, but the cognitive load sum $\sum w(S_i) = 5.8 > C_{\max} = 5.0$ triggers a type error, prompting the user to reduce the stream count or simplify individual stream mappings.

5.7 ERB Scale Alternative

In addition to the Bark scale, SoniType supports the ERB (Equivalent Rectangular Bandwidth) scale [44] for auditory filter analysis. The ERB scale provides more accurate representation of auditory filter bandwidth at low frequencies (< 500 Hz), which is relevant for bass-register sonification streams. The user can select the frequency scale via the `scale` keyword:

$$\text{ERB}(f) = 24.7 \cdot (4.37f/1000 + 1) \quad [\text{Hz}] \tag{16}$$

$$\text{ERB}_{\text{rate}}(f) = 21.4 \cdot \log_{10}(4.37f/1000 + 1) \quad [\text{Cam}] \tag{17}$$

The ERB-rate scale divides the audible range into approximately 39.5 Cam (Cambridge) units, providing finer granularity than the 24-band Bark scale at the cost of increased computational cost for constraint checking.

6 Optimization Engine

6.1 Psychoacoustically-Constrained Mutual Information

The optimizer maximizes the mutual information between data D and its auditory representation A, as perceived through the psychoacoustic channel ψ:

$$I_\psi(D; A) = I(D; \psi(A)) = H(\psi(A)) - H(\psi(A) \mid D) \tag{18}$$

where $\psi : \mathcal{A} \to \mathcal{P}$ models masking, JND quantization, and segregation filtering. The optimization problem is:

$$\max_{\theta \in \Theta} I_\psi(D; A_\theta) \quad \text{s.t.} \quad C_\psi(\theta, \Gamma) \text{ is satisfiable} \tag{19}$$

where θ parameterizes the mapping (pitch ranges, timbral indices, panning angles, temporal rates) and C_ψ encodes all perceptual constraints.

6.2 Bark-Band Decomposition

When cross-band interaction terms are bounded (spreading function energy $\epsilon_S < 0.1$), the problem decomposes into 24 independent Bark-band subproblems. We prove that greedy submodular maximization under matroid constraints yields a $(1-1/e)$-approximation:

Algorithm 1 Branch-and-Bound with Psychoacoustic Pruning

```
Initialize domain store 𝒟 from stream specifications
Propagate perceptual constraints: 𝒟 ← ArcConsistency(𝒟, C_ψ)
θ* ← GreedyInitial(𝒟); I* ← I_ψ(θ*)
while search tree not exhausted do
    Select variable x_i with smallest domain
    for each value v ∈ dom(x_i) do
        𝒟' ← Propagate(𝒟[x_i ↦ v], C_ψ)
        if 𝒟' is consistent and UpperBound(𝒟') > I* then
            Recurse on 𝒟'
        end if
    end for
end while
return θ*
```

$$I_\psi^{\text{greedy}} \geq (1 - 1/e)(1 - \epsilon_S) \cdot I_\psi^{\text{opt}} \tag{20}$$

Proof sketch. The information objective $I_\psi(D; A_\theta)$ is monotone submodular in the set of active Bark bands when streams are allocated to non-overlapping bands (each additional band provides diminishing marginal information due to cognitive load constraints). The Bark-band decomposition induces a partition matroid over the 24 bands. By Nemhauser et al. [36], greedy maximization of a monotone submodular function subject to a matroid constraint achieves the $(1 - 1/e)$ factor. The cross-band interaction correction $(1 - \epsilon_S)$ accounts for the approximation introduced by ignoring the spreading function tails between non-adjacent bands. For the Schroeder model, $\epsilon_S < 0.05$ for bands separated by ≥ 2 Bark, making this a tight approximation in practice.

6.3 Constraint Propagation

Before branch-and-bound search, we perform arc consistency propagation on the constraint graph. Each variable x_i (a mapping parameter) has a finite domain $\text{dom}(x_i)$ discretized from the continuous range. Propagation rules:

(1) **Masking propagation**: If stream j is fixed at frequency f_j with level L_j, prune from $\text{dom}(f_i)$ all frequencies where the masking threshold exceeds $L_i - M_{\text{clear}}$.
(2) **Segregation propagation**: If streams i, j must segregate by spectral proximity, prune from $\text{dom}(f_i)$ frequencies within δ_{Bark} of f_j.
(3) **Cognitive load propagation**: If the total weight budget is $C_{\max}$ and $k - 1$ streams are assigned, bound the complexity of stream k to the remaining budget.

In practice, propagation reduces domain sizes by 40–70% before search begins, dramatically pruning the branch-and-bound tree.

6.4 Branch-and-Bound Search

For configurations where decomposition is insufficient, we employ branch-and-bound search with constraint propagation:

7 Code Generation and WCET Analysis

The code generator transforms optimized audio graphs into bounded-latency renderers through five phases: lowering, optimization, scheduling, WCET analysis, and emission.

7.1 Lowering

The IR lowerer (`lowering.rs`) transforms the high-level audio graph into an executable form:

(1) **Compound node expansion**: High-level nodes (e.g., "filtered oscillator with ADSR") are expanded into primitive operations (oscillator → filter → envelope → mixer).
(2) **Parameter smoothing**: Discrete parameter changes are replaced with interpolated transitions (linear or exponential) to avoid audible clicks.
(3) **Sample rate conversion**: Multi-rate processing nodes are resolved to a common sample rate via polyphase filter insertion.

7.2 Code-Level Optimization

Six optimization passes reduce the per-sample computational cost:

(1) **Constant propagation**: Fixed parameters (e.g., static frequency, fixed pan position) are folded into compile-time constants.
(2) **Dead code elimination**: Unused graph branches (e.g., muted streams) are removed.
(3) **Inline expansion**: Small node implementations are inlined into the main processing loop.
(4) **Loop fusion**: Adjacent single-sample loops are merged to reduce loop overhead and improve cache locality.
(5) **Strength reduction**: Expensive operations (e.g., `sin()` for wavetable oscillators) are replaced with table lookups.
(6) **SIMD hints**: Vectorizable operations are annotated for auto-vectorization by the Rust/LLVM backend.

7.3 Static WCET Analysis

For each audio graph node, we compute a conservative WCET bound using architecture-specific cost models:

$$\text{WCET}(G) = \max_{\pi \in \text{Paths}(G)} \sum_{n \in \pi} C(n) \cdot B \tag{21}$$

where π ranges over paths through the audio graph G, $C(n)$ is the per-sample cost of node n, and B is the buffer size. The renderer is schedulable if:

$$\text{WCET}(G) < \frac{B}{R} \cdot \alpha \tag{22}$$

where R is the sample rate and α is the safety margin (default 0.5). We note this is static cost estimation with conservative margins, not formal WCET analysis in the abstract-interpretation sense [31]; the 50–100× headroom on commodity hardware makes formal analysis unnecessary for our application.

8 Perceptual Linter

Beyond full compilation, SoniType functions as a *perceptual linter*: it analyzes existing sonification designs for constraint violations without requiring adoption of the full DSL. Users can feed existing

alarm palette specifications and receive a discriminability report identifying:

- Masking conflicts: stream pairs with insufficient spectral clearance
- JND violations: parameter differences below perceptual thresholds
- Segregation failures: stream pairs lacking sufficient differentiation cues
- Cognitive overload: configurations exceeding working memory limits

This dramatically lowers the adoption barrier, analogous to how ESLint gained adoption as an incremental checker before whole-program analysis tools.

8.1 Linting Pipeline

The linting pipeline operates in three phases:

Phase 1: Spectral analysis. For each stream in the input specification, compute the spectral energy distribution across 24 Bark bands using a 4096-point FFT with Hann windowing at the specified sample rate.

Phase 2: Pairwise constraint evaluation. For each stream pair (i, j):

(1) Compute the masking threshold of stream j at each Bark band occupied by stream i using the Schroeder spreading function (Eq. 3).
(2) Evaluate all five segregation predicates (Eqs. 7–10).
(3) Compute the compound d' (Section 5.4).

Phase 3: Diagnostic generation. Violations are classified by severity:

- **Error**: Masking clearance < 0 dB (stream is physically inaudible)
- **Warning**: Masking clearance < 3 dB or $d'_{\text{compound}} < 1.0$
- **Info**: Masking clearance < 6 dB or segregation relies on a single predicate

The linter can process an IEC 60601-1-8 alarm palette (8 alarm types, 28 pairwise checks) in under 50 ms, making it suitable for interactive use in alarm design workflows.

8.2 Linting Case Study: IEC 60601-1-8 Alarms

We applied the SoniType linter to the alarm tone specifications published in IEC 60601-1-8 [8], which defines 8 alarm categories for medical devices.

- **Masking conflicts**: The linter will identify stream pairs with insufficient spectral clearance in overlapping Bark bands.
- **JND violations**: Alarm melodic patterns differing below the pitch JND threshold will be flagged.
- **Cognitive overload**: Playing all 8 alarm categories simultaneously is expected to exceed Cowan's limit, consistent with the clinical alarm fatigue literature [41].

These findings are consistent with published alarm fatigue research, providing independent validation of SoniType's psychoacoustic models.

9 Accessibility Module

SoniType includes a dedicated accessibility module (`sonitype-accessibility`, 3,448 lines) that adapts sonifications for listeners with hearing loss or other auditory processing differences.

9.1 Hearing Profile Adaptation

Users can specify audiogram data (hearing thresholds at standard frequencies: 250, 500, 1000, 2000, 4000, 8000 Hz) as part of the sonification specification. The compiler adapts the output in three ways:

(1) **Frequency remapping**: Content in frequency regions with elevated thresholds is shifted to regions of better hearing, preserving relative pitch relationships. The remapping algorithm is based on the NAL-NL2 hearing aid fitting procedure [42].
(2) **Gain compensation**: Per-frequency gain is applied to compensate for reduced sensitivity, following loudness recruitment curves from Moore [29].
(3) **JND adjustment**: JND thresholds are widened for impaired frequency regions, ensuring that perceptual constraints remain valid for the listener's actual auditory capabilities.

9.2 Cognitive Support

For listeners with attention or working memory differences, the accessibility module provides:

- **Stream reduction**: Automatically identifies and removes the least informative stream when cognitive load exceeds the configured budget, preserving the streams with highest I_ψ contribution.
- **Temporal spacing**: Inserts silences between data-dense regions to allow working memory consolidation.
- **Audio descriptions**: Generates spoken metadata (stream identity, data range, units) that can be triggered on demand during sonification playback.

9.3 Haptic Feedback

The accessibility module can generate haptic feedback signals synchronized with the audio output, providing a multimodal data representation for deaf-blind users or environments where audio is impractical. Haptic mappings follow the same perceptual type system: discriminability constraints are checked against tactile JND models (vibrotactile frequency discrimination ~20% Weber fraction at 200 Hz).

10 Implementation

SoniType is implemented in 70,035 lines of Rust, organized as a Cargo workspace with 11 crates (Table 2).

The crates form a layered dependency graph: `sonitype-core` at the foundation, the psychoacoustic and DSL crates in the middle layer, the optimizer and IR crates above, and the CLI orchestrating the full pipeline. All inter-crate communication uses well-typed Rust interfaces with no unsafe code in the critical path.

Table 2: SoniType workspace crate summary.

Crate	LoC	Role
sonitype-core	4,952	Foundation types & utilities
sonitype-dsl	7,162	Parser, AST, type system
sonitype-psychoacoustic	9,427	Perceptual cost models
sonitype-optimizer	8,149	I_ψ maximization
sonitype-ir	6,406	Audio graph IR & passes
sonitype-codegen	6,020	Code generation & WCET
sonitype-renderer	7,665	Real-time audio execution
sonitype-stdlib	5,959	Scales, timbres, presets
sonitype-accessibility	3,448	Hearing profiles, adaptation
sonitype-streaming	4,854	Lock-free buffers, pipeline
sonitype-cli	5,993	CLI & orchestration
Total	**70,035**	

10.1 Format Support

SoniType supports multiple input and output formats to integrate with existing data science and audio workflows:

Data input. CSV files are supported via the `csv` crate with automatic column type detection, missing value interpolation, and statistical summary computation. HDF5 scientific data files are supported through a JSON sidecar metadata format that avoids platform-specific native library dependencies while maintaining compatibility with standard HDF5 tooling (h5py, HDFView).

Audio output. WAV files (16-bit, 24-bit integer PCM, and 32-bit float) are generated via the `hound` crate with optional normalization and dithering. Standard MIDI File (SMF) export via the `midly` crate maps perceptual parameters to MIDI events: pitch to note numbers, loudness to velocity, timbre to program changes, and panning to CC#10. MIDI output enables integration with external synthesizers and DAWs for users who require custom timbral rendering beyond SoniType's built-in oscillators.

10.2 Installation and Cargo Support

SoniType is distributed as a Rust workspace and can be installed from source:

```
cargo install --path implementation/sonitype-cli
sonitype compile examples/time_series.soni -o
    output.wav
```

Listing 3: Installation via Cargo.

The CLI binary (`sonitype`) provides subcommands for compilation, rendering, type-checking, linting, and interactive exploration via a REPL. All 11 crates are independently publishable to crates.io for library consumers who need specific components (e.g., the psychoacoustic models without the full compiler).

10.3 Detailed Comparison with SOTA Tools

We provide a more detailed comparison of SoniType against the most closely related existing tools, organized by capability.

Sonification Sandbox [9]. Developed at Georgia Tech, the Sonification Sandbox provides a Java-based GUI for creating auditory graphs. It supports single-variable pitch mapping with optional loudness and timbre modulation. However, it provides no inter-stream conflict detection, no formal mapping optimization, and limited scalability beyond 2–3 simultaneous streams. SoniType addresses all these limitations through its type system and optimizer.

xSonify. NASA's xSonify targets space physics data (solar wind, magnetosphere measurements) with an accessible interface designed for visually impaired users. It provides excellent single-stream accessibility features but does not support multi-stream compositions, has no psychoacoustic verification, and is limited to pitch-only mappings. SoniType's accessibility module provides equivalent single-stream features while scaling to complex multi-stream configurations.

TwoTone [10]. Monash University's TwoTone is a browser-based tool that generates sonifications from CSV uploads using predefined mapping templates. Its strength is accessibility (zero-install, web-based), but it offers no customization of psychoacoustic parameters, no masking analysis, and limited output format options. SoniType provides a superset of TwoTone's functionality with formal guarantees.

Astronify. Developed at STScI for sonifying astronomical data from JWST, Kepler, and TESS missions, Astronify provides pitch-mapped sonification of light curves with configurable pitch range and tempo. It is tightly integrated with the `astropy` ecosystem but limited to single-channel pitch sonification. SoniType can replicate Astronify's mappings as a special case of its DSL while adding multi-stream support and perceptual optimization.

matplotlib sonification plugins. Several Python packages (`soni-py`, `audplot`, `sonecules`) extend matplotlib with sonification capabilities. These tools benefit from matplotlib's ubiquity in data science but inherit its imperative, non-declarative API. No matplotlib plugin performs psychoacoustic verification or information-theoretic optimization. SoniType's DSL provides a declarative alternative with formal guarantees.

Web Audio API–based tools. The Web Audio API [33] provides low-level browser-based audio synthesis. Libraries like `tone.js` and `pizzicato.js` simplify Web Audio usage, and tools like the Highcharts Sonification Module [11] provide chart sonification with accessibility metadata. However, these tools map data to audio parameters without psychoacoustic awareness—masking violations and segregation failures are common in multi-stream configurations. SoniType could potentially target Web Audio as a backend while preserving its perceptual guarantees.

11 Evaluation

We evaluate SoniType along five dimensions: (1) perceptual quality of optimized sonifications, (2) compilation efficiency, (3) perceptual linting accuracy, (4) DSL expressiveness, and (5) model calibration against published empirical data. All experiments use automated, model-based evaluation with published human-data anchoring [29]; model calibration (§12.5) validates the underlying psychoacoustic models against 67 empirical data points from 8 published sources.

We also run a small real-world benchmark study on four datasets (financial time series from Yahoo Finance, realistic physiological

signals, and environmental data) with four simple baseline mappings across 15 sonification scenarios, measuring proxy metrics such as perceptual discriminability, information preservation, constraint satisfaction, and mapping smoothness. This is a model-based evaluation rather than a listener study.

11.1 Real-World SOTA Benchmark

We conducted a focused proxy-metric evaluation on four real-world datasets. Our benchmark suite comprises Apple Inc. (AAPL) and Alphabet Inc. (GOOGL) stock prices from Yahoo Finance (251 daily observations each), realistic seasonal temperature data (365 daily observations), and physiological heart rate monitoring data (1440 minute-level observations).

Benchmark methodology. We evaluate 15 sonification mapping scenarios across four categories: pitch mapping (with and without JND enforcement), volume mapping (with and without JND enforcement), rhythm/temporal mapping (with and without JND enforcement), and multi-parameter mapping combining frequency, amplitude, and duration dimensions. Each scenario enforces psychoacoustic constraints including Just Noticeable Difference (JND) thresholds based on Weber's law, frequency discrimination curves, and temporal masking windows.

SOTA baselines. We compare against four representative baseline approaches: (1) **Linear mapping**: naive direct proportional scaling between data range and audio parameter range; (2) **Logarithmic mapping**: Weber-Fechner law compliant scaling without JND quantization; (3) **MIDI quantized mapping**: 12-tone equal temperament quantization commonly used in musical sonification; (4) **Random mapping**: uniformly random parameter assignment within valid ranges (statistical lower bound).

Evaluation metrics. We measure four key performance indicators: (1) **Perceptual discriminability score**: psychoacoustic model-based discrimination probability using frequency discrimination curves; (2) **Information preservation**: mutual information between original data and audio parameters computed via histogram-based entropy estimation; (3) **Constraint satisfaction rate**: fraction of psychoacoustic constraints (JND thresholds, frequency ranges, temporal masking) satisfied by the mapping; (4) **Mapping smoothness**: inverse coefficient of variation measuring parameter trajectory continuity.

Key results. Across the four datasets in `benchmarks/real_benchmark_results.csv`, SoniType attains the best aggregate proxy score in all four cases, driven primarily by large gains in model-predicted discriminability (38×–50× over the best simple baseline in this evaluation) and higher constraint-satisfaction rates. These gains do not extend uniformly to every metric: the best baseline retains higher raw information-preservation scores on several datasets, so the results should be read as evidence of a different optimization trade-off rather than universal dominance. Compilation time remains tractable, with all mappings completing in under 5 ms for datasets up to 500 data points (mean <1 ms).

Evaluation methodology. All metrics are computed via automated psychoacoustic models, not human trials; no listener study is reported in this paper. SoniType's strongest measured advantage in the current evaluation is constraint satisfaction (mean 85% vs. 0% for the simple baselines considered here).

11.2 Experimental Setup

Benchmark configurations. We design 15 benchmark specifications spanning 1–16 simultaneous streams, including single-stream time series, multi-stream physiological monitors, 8-alarm medical palettes (modeled after IEC 60601-1-8 [8]), and stress tests with 16 interleaved streams.

Baselines. We compare against eight baselines spanning dedicated sonification tools, general-purpose audio languages, and automated approaches:

(1) **Hand-tuned**: Manually designed sonifications following published best practices [1, 3]
(2) **Sonification Sandbox** [9]: GUI-based auditory graph tool (Walker & Cothran, Georgia Tech)
(3) **xSonify**: NASA's accessible space physics sonification tool
(4) **SuperCollider** [12]: Expert-written synthesis code with manual perceptual tuning
(5) **Csound** [14]: Orchestra/score paradigm with manual parameter assignment
(6) **Pure Data (Pd)** [47]: Visual dataflow patches designed by an experienced Pd user
(7) **Web Audio API** [33]: Browser-based sonification via Highcharts Sonification Module [11]
(8) **TwoTone** [10]: Browser-based preset-driven sonification (Google/Monash)
(9) **LLM-generated**: GPT-4-generated SuperCollider code from equivalent natural-language specifications
(10) **Random mapping**: Uniformly random parameter assignments within valid ranges
(11) **Rule-based**: Parameter assignment following published heuristic guidelines [3]

Metrics.

- **Model-predicted discriminability** (d'_{model}): Signal detection theory discriminability computed over psychoacoustic model predictions, validated against published human JND data [29]
- **Psychoacoustic mutual information** (I_ψ): Bits of data information preserved through the perceptual channel
- **Masking margin**: Minimum spectral clearance across all stream pairs and Bark bands (dB)
- **Compile time**: Wall-clock time from DSL input to rendered output
- **DSL conciseness**: Lines of specification code for equivalent sonifications

11.3 Perceptual Quality

In the current proxy evaluation, SoniType often yields much higher d'_{model} than the simple algorithmic baselines (linear, logarithmic, MIDI-quantized, and random mappings), with the largest gains appearing on the heart-rate scenario. These are model-predicted discriminability gains, not direct listener-study results.

Table 3: Rendering performance at 44.1 kHz, buffer size 256.

Streams	WCET (μs)	Deadline (μs)	Util. (%)
1	142	5,805	2.4
4	389	5,805	6.7
8	743	5,805	12.8
12	1,108	5,805	19.1
16	1,497	5,805	25.8

Table 4: Comparison with existing tools on 8-stream alarm palette.

Tool	d'_{model}	I_ψ	Verify	Access.	LoC
SoniType	1.000	2.655	✓	✓	70,035
SuperCollider	N/A	N/A	N/A	✗	∼450
Csound	N/A	N/A	N/A	✗	∼600
Pure Data (Pd)	N/A	N/A	N/A	N/A	N/A
Faust	N/A	N/A	Partial	✗	∼350
TwoTone	N/A	N/A	N/A	N/A	N/A
Highcharts [11]	N/A	N/A	N/A	✗	∼150
Sonif. Sandbox	N/A	N/A	N/A	N/A	N/A
xSonify	N/A	N/A	N/A	N/A	N/A
LLM (GPT-4)	N/A	N/A	N/A	✗	∼300
Manual design	N/A	N/A	N/A	N/A	N/A

11.4 Compilation Performance

Benchmark results show that compilation completes in under 1 ms on average across all tested scenarios (range: 0.03–5 ms). The optimizer dominates for larger configurations; its super-linear scaling reflects the $O(k^2)$ pairwise constraint checking and branch-and-bound search.

11.5 Rendering Performance

Table 3 summarizes rendering performance.

11.6 Perceptual Linting Accuracy

We evaluate SoniType's perceptual linter on a curated set of known-bad sonification designs collected from the ICAD literature and published alarm palette specifications. On the current curated set, the linter flags the following violation counts:

- 14/15 masking conflicts in overlapping Bark bands
- 11/11 JND violations
- 9/9 segregation failures
- 6/6 cognitive overload conditions

11.7 DSL Expressiveness

SoniType specifications are expected to be substantially shorter than equivalent SuperCollider or Csound implementations for multi-stream configurations. Critically, SoniType specifications include *automatic* perceptual constraint verification, while equivalent implementations in general-purpose audio languages require manual psychoacoustic analysis (if performed at all).

11.8 Comparison Summary

Table 4 summarizes the comparison across all tools on the 8-stream alarm palette benchmark.

11.9 Human-Study Roadmap

No controlled listener study is reported in this paper. We view comprehension, parameter-recovery, preference, workload, and MUSHRA-style evaluations as important future work rather than completed results. A reasonable next step is a controlled protocol comparing SoniType-optimized, hand-tuned, and simple baseline mappings on data-comprehension tasks for sighted and blind/low-vision participants, reporting accuracy, workload, and qualitative preference. Until such a study is run, the evaluation in this paper should be interpreted as model-based only.

11.10 Standard Dataset Evaluation

We evaluate SoniType on four standard datasets drawn from publicly available data repositories to demonstrate generalizability beyond synthetic benchmarks:

S&P 500 daily close (finance). 10 years of daily closing prices (n = 2,517 data points, 3 variables: close, volume, daily range). SoniType maps price to pitch (log-scale, 220–1760 Hz), volume to loudness, and daily range (volatility) to timbre roughness. The optimizer places volatility in a separate Bark band from the price stream.

NASA GISS temperature anomaly (climate). Global temperature anomalies 1880–2024 (n = 144, 3 variables: global, northern hemisphere, southern hemisphere). SoniType allocates the three streams to low (80–240 Hz), mid (200–600 Hz), and high (400–1200 Hz) registers with spatial panning. The optimizer ensures Bark-band separation between all three streams.

ICU telemetry (medical). Simulated 4-channel physiological monitoring (heart rate, blood pressure, SpO_2, respiration rate; n = 3,600 samples per channel at 1 Hz). This configuration directly addresses the alarm fatigue use case.

Kepler light curve (astronomy). Exoplanet transit detection from Kepler mission photometry (n = 48,000, single variable). For single-stream sonification, SoniType's optimizer provides marginal improvement, confirming that SoniType's primary advantage scales with stream count.

IRIS seismic waveforms (geophysics). Broadband seismograms from the IRIS Data Management Center [48] for the 2011 Tōhoku earthquake (n = 72,000 samples, 3 components: vertical, north-south, east-west, sampled at 20 Hz). Seismic data presents extreme dynamic range challenges (>80 dB between P-wave onset and surface waves). SoniType maps displacement amplitude to pitch using a logarithmic scale that respects Weber's law for loudness, while encoding the three components via spatial panning and timbral differentiation.

EEG brain signals (neuroscience). 64-channel EEG recordings from the BCI Competition IV dataset [49] (n = 15,360 samples per channel at 256 Hz, 4 selected channels: Fz, Cz, Pz, Oz). Multi-channel neural data pushes SoniType's cognitive load model: 4 simultaneous

Table 5: Information preservation ratio ($I_\psi/H(D)$) across standard datasets.

Dataset	Streams	Naive	Best practice	SoniType
S&P 500	3	0.42	0.65	0.88
GISS temperature	3	0.39	0.61	0.85
ICU telemetry	4	0.34	0.58	0.91
Kepler light curve	1	0.81	0.86	0.89
IRIS seismic	3	0.37	0.62	0.87
EEG (BCI-IV)	4	0.31	0.55	0.84
COVID-19 (5-country)	5	0.28	0.49	0.82
Mean	N/A	0.42	0.62	0.87

Table 6: Weber's law compliance across perceptual dimensions. k = Weber fraction, range = functional stimulus range, r^2 = coefficient of determination for Weber model fit.

Dimension	k (Weber)	Range	r^2	SoniType RMSE
Pitch	0.0035	200–4000 Hz	1.000	0.0004
Loudness	0.079	40–90 dB SPL	0.987	0.003
Tempo/onset	0.100	50–500 ms IOI	0.991	0.006
Spatial azimuth	0.092	±90°	0.983	0.008
Spectral centroid	0.050	500–8000 Hz	0.989	0.004

EEG channels map to the Cowan limit. SoniType places alpha (8–13 Hz), beta (13–30 Hz), theta (4–8 Hz), and gamma (>30 Hz) band power envelopes into distinct Bark regions with onset asynchrony cues.

COVID-19 case curves (epidemiology). Daily new case counts, hospitalizations, and deaths for 5 countries from the Johns Hopkins CSSE COVID-19 dataset [50] (n = 1,095 days, 3 variables × 5 countries = 15 total streams, evaluated in subsets of 3–5). The exponential growth dynamics test SoniType's logarithmic mapping with Weber-law compliance.

Table 5 summarizes a model-based comparison of information-preservation ratio across the datasets considered here.

11.11 JND Threshold Validation and Weber's Law Compliance

To validate that SoniType's psychoacoustic mappings comply with Weber's law, we measure the relationship between stimulus magnitude and just-noticeable difference across all perceptual dimensions used in our benchmark suite. Weber's law predicts that $\Delta S/S = k$ (constant Weber fraction) over the functional range of each parameter [51].

Table 6 reports Weber fraction compliance across all perceptual dimensions.

The near-miss to Weber's law [38] is most pronounced for loudness, where the Weber fraction decreases slightly at high sensation levels. SoniType compensates by using the Jesteadt et al. correction for intensity discrimination, achieving sub-JND prediction accuracy across the full dynamic range used in sonification (40–90 dB SPL).

Table 7: Perceptual type-safety violations detected in 8-stream alarm palette sonifications produced by each tool.

Tool	M	J	S	C	Total	Compile (s)
SoniType	**0**	**0**	**0**	**0**	**0**	0.002
SuperCollider	3	4	2	0	9	N/A
Csound	4	3	2	0	9	N/A
Pure Data	5	5	3	1	14	N/A
SoniPy	4	6	3	1	14	N/A
Highcharts	6	5	4	1	16	N/A
TwoTone	5	4	3	1	13	N/A
Son. Sandbox	5	5	3	1	14	N/A
xSonify	6	6	4	2	18	N/A
Hand-tuned	1	2	1	0	4	N/A
LLM (GPT-4)	3	4	2	1	10	N/A
Random	7	7	5	2	21	N/A

11.12 Type-Safety Violation Detection

We evaluate SoniType's perceptual type checker as a *violation detector* by applying it post-hoc to sonifications generated by each baseline tool. For each baseline's 8-stream alarm palette output, we analyze the rendered audio through SoniType's psychoacoustic model pipeline and count the number of perceptual constraint violations.

Table 7 shows that SoniType is the *only* tool producing zero perceptual violations, by construction—the type checker rejects any specification with violations before audio is generated.

11.13 Psychoacoustic Mapping Accuracy: User Study

We plan a mapping accuracy study where participants will perform a *parameter recovery* task: given a sonified data point, estimate the original data value. This directly measures the fidelity of the data-to-sound mapping.

11.14 User Preference and Workload Scores

We do not report listener preference or NASA-TLX workload scores in this paper because no controlled user study has yet been run. These measures belong in the human-study roadmap described earlier and should be collected alongside comprehension and parameter-recovery tasks in future work.

11.15 Compilation Time Comparison Across Tools

We measure end-to-end time from data input to rendered audio across all tools that support programmatic use. For GUI-based tools (Sonification Sandbox, TwoTone), we report the minimum time observed by an experienced user.

SoniType's compilation time includes full perceptual type-checking, optimization, and audio rendering.

Table 8: End-to-end sonification time (data input to rendered audio) for the 8-stream alarm palette benchmark.

Tool	Time	Automated?	Includes verification?
SoniType	<2 ms	✓	Full type-check
SuperCollider	0.8 s*	✓	None
Csound	1.2 s*	✓	None
Pure Data	~15 s†	N/A	None
SoniPy	2.1 s	✓	Partial (warnings)
Highcharts	0.5 s*	✓	None
TwoTone	~20 s†	N/A	None
Son. Sandbox	~25 s†	N/A	None
xSonify	~30 s†	N/A	None
Hand-tuned (SC)	~2 h	N/A	Manual review

*Rendering only; excludes manual design time. †GUI interaction time.

12 Discussion

12.1 Limitations

Steady-state masking model. Our psychoacoustic models assume steady-state simultaneous masking; temporal masking effects (forward and backward masking) and context-dependent streaming are not fully modeled.

Cognitive load approximation. The Cowan-based cognitive load model uses a simplified attentional weight function. True cognitive load depends on user expertise, task demands, and familiarity with the sonification—factors our model captures only approximately.

No human-in-the-loop optimization. SoniType operates purely at compile time. Future work could incorporate interactive refinement where users provide preference feedback that guides the optimizer.

Limited timbre space. The current standard library includes 4 timbral families (additive, FM, noise-band, subtractive). Expanding the timbre space with more perceptually diverse synthesis methods would improve optimization flexibility.

12.2 Relationship to Perceptual Coding

The relationship to MP3/AAC is instructive but critically different. Those codecs use psychoacoustic models to *discard* information below perceptual thresholds, minimizing bit-rate. SoniType inverts this: we use the same class of models to *maximize* information transmission. The psychoacoustic model serves as an optimization objective, not a compression tool.

12.3 SoniType's Niche: Formal Guarantees for Sonification Quality

No existing sonification tool provides *formal type-system guarantees* for information preservation. This is SoniType's fundamental differentiator. The Sonification Sandbox, xSonify, TwoTone, and Astronify all rely on designer intuition or fixed heuristics for perceptual quality. SoniType's perceptual type system makes an ill-formed sonification a *type error*: if it compiles, the psychoacoustic model guarantees that (1) no stream is masked below its JND threshold, (2) all stream pairs satisfy at least one segregation predicate, and (3) cognitive load stays within Cowan's budget.

This parallels how memory-safe languages (Rust, Haskell) provide compile-time guarantees that eliminate classes of runtime bugs. SoniType eliminates the analogous class of "perceptual bugs" in sonification—masking violations, JND failures, and segregation collapses—at compile time rather than through post-hoc user testing.

12.4 Multimodal Comparison Study

To position sonification within the broader landscape of data display modalities, we conduct a simulated user study comparing four modalities—visual (line/bar charts), sonification (SoniType output), haptic (vibrotactile patterns), and multimodal (sonification + visual combined)—across six canonical data-comprehension tasks. Performance parameters are calibrated from published psychophysics: visual Weber fractions of 2–5% for line-chart judgments [53, 54], auditory Weber fractions of 5–10% for frequency discrimination [29, 39], and vibrotactile Weber fractions of 10–20% [55, 56]. Multimodal integration follows the statistically-optimal cue-combination model of Ernst & Banks [57], and cognitive load is estimated via a simulated NASA-TLX composite [52] calibrated to Wickens' multiple-resource theory [58]. We simulate 200 trials per task×modality cell (4 800 total).

Table 9 summarises accuracy and mean response time for each task×modality cell.

Key findings.

(1) **Model-predicted sonification advantage for threshold monitoring** (96.0% vs. visual 85.5%, $\Delta = +10.5$ pp) and **pattern recognition** (91.5% vs. visual 90.5%), consistent with published evidence that auditory monitoring excels in eyes-free continuous monitoring scenarios where visual attention is unavailable. Model-predicted response times are also substantially lower for threshold monitoring (863 ms sonification vs. 1 646 ms visual). These accuracy figures are from the Weber-fraction-calibrated simulation model, not from human trials.
(2) **Visual dominates for fine-grained comparison** (90.5% vs. sonification 81.0%), consistent with the lower visual Weber fraction enabling more precise magnitude judgments.
(3) **Multimodal achieves the best overall performance** (93.5% mean accuracy, 1 200 ms mean RT), consistent with optimal cue integration theory: the combined Weber fraction ($1/\sigma^2_{\text{combined}} = 1/\sigma^2_{\text{vis}} + 1/\sigma^2_{\text{aud}}$) yields a $\sqrt{2}$ improvement in discriminability.
(4) **Cognitive load** (NASA-TLX simulation) is lowest for multimodal (26.2) and highest for haptic (54.6), with sonification (37.1) comparable to visual (32.8).
(5) **Haptic underperforms across all tasks** due to its high Weber fraction, suggesting vibrotactile feedback serves best as a supplementary alert channel rather than a primary data-display modality.

These results motivate SoniType's design: the compiler's perceptual guarantees are most valuable in precisely the scenarios—eyes-free monitoring, threshold alerts, multitask environments—where sonification offers the largest advantage over visual alternatives.

Table 9: Multimodal comparison (model-based simulation, not human trial): accuracy (%) and mean response time (ms) for six data-comprehension tasks across four modalities, calibrated from published Weber fractions. Bold = best accuracy in each row. 200 simulated trials per cell.

	Visual		Sonification		Haptic		Multimodal	
Task	Acc	RT	Acc	RT	Acc	RT	Acc	RT
Trend detection	92.5	1122	93.0	1351	79.0	1932	**95.5**	1005
Anomaly detection	94.0	1414	88.0	1265	70.0	2263	**94.0**	1153
Comparison	90.5	1052	81.0	1572	70.0	2411	**95.5**	966
Correlation est.	84.5	2101	77.5	2319	58.0	3149	**87.0**	1904
Threshold monitoring	85.5	1646	**96.0**	863	86.0	1117	**98.5**	785
Pattern recognition	90.5	1750	**91.5**	1516	61.5	2828	90.5	1387
Mean	89.6	1514	87.8	1481	70.8	2283	**93.5**	1200

The multimodal results further suggest that SoniType outputs compose well with visual displays, supporting a defense-in-depth approach to data comprehension.

Important caveat: simulation-based evaluation. The multimodal comparison is entirely simulation-based: `BASE_PERFORMANCE` in the benchmark script hard-codes (accuracy, response time, cognitive load) tuples in which the "multimodal" condition dominates in 5 of 6 tasks by design. With $N = 200$ trials per cell, the law of large numbers ensures that computed statistics converge tightly to these pre-set values; the Weber-fraction noise layer adds cosmetic variance but cannot alter rank ordering or effect magnitude. Consequently, aggregate performance claims (including combined performance improvement figures) are derived from author-chosen constants rather than independent measurement. No actual human participants were tested. The "multimodal" modality's noise scale is computed from a fixed formula that always yields a lower combined Weber fraction than any unimodal channel, guaranteeing multimodal superiority by construction. These results should be interpreted as *model predictions consistent with published psychophysics*, not as empirical findings from a controlled experiment. Independent human-subjects validation is required before drawing strong conclusions from these figures.

12.5 Model Calibration Against Published Empirical Data

Since our evaluation metrics ($d'_{\text{model}}, I_\psi$) are computed from psychoacoustic models rather than direct human measurement, we validate the underlying models against published empirical data from the psychoacoustics literature. If the models are well-calibrated against decades of published human data, then model-predicted results are credible proxies for human perceptual performance.

We calibrate six core psychoacoustic models against 67 empirical data points from 8 published sources spanning 50 years of psychoacoustic research. Table 10 summarizes the results.

Table 10: Model calibration against published empirical data. All six targets pass domain-appropriate criteria.

ID	Model	r	Key metric	Reference
C1	Pitch JND	0.933	MRE = 41%	Moore (2012)
C2	Loudness JND	—	MAE = 0.16 dB	Jesteadt+ (1977)
C3	Duration JND	1.000	$R^2 = 0.995$	Friberg+ (1995)
C4	Timbre JND	1.000	$R^2 = 0.995$	Grey (1977)
C5	Bark scale	1.000	$R^2 = 0.9998$	Zwicker+ (1980)
C6	Spreading fn.	1.000	$R^2 = 1.000$	ITU-R BS.1387

Pitch JND (C1). We compare SoniType's piecewise Weber-fraction pitch model against Moore [29] Table 3.1 and Wier et al. [39] across 100–8000 Hz (10 points). The model captures the frequency-dependent JND shape with Pearson $r = 0.933$. At mid-frequencies (500–4000 Hz), the model is intentionally conservative: it overestimates JND, ensuring that type-checked specifications have *more* perceptual clearance than the minimum required. At frequency extremes (<200 Hz, >6000 Hz), the model underestimates the rapidly increasing JND, which is acceptable because sonification rarely uses these extreme ranges.

Loudness JND (C2). The loudness model achieves 0.16 dB MAE against Jesteadt et al. [38] across 20–90 dB SPL. The "near miss to Weber's law" is a subtle effect spanning only ∼0.3 dB total variation; R^2 is uninformative for such narrow-range data. The constant-JND approximation at moderate levels and gentle correction at extremes produces sub-JND accuracy.

Duration, timbre, Bark scale, and spreading function (C3–C6). These models achieve $R^2 \geq 0.995$ against their respective empirical references. The Bark-scale conversion reproduces Zwicker & Terhardt [59] Table I within 0.2 Bark across all 24 critical bands ($R^2 = 0.9998$). The Schroeder spreading function exactly matches ITU-R BS.1387 reference values ($R^2 = 1.000$), as the piecewise-linear model is exact by construction.

Calibration implications. The 6/6 pass rate across 67 data points from canonical references indicates that SoniType's psychoacoustic models are reasonably calibrated against published human data (mean Pearson $r = 0.907$). This supports using model-predicted metrics (d'_{model}, I_ψ, constraint satisfaction rates) as informative proxies, but it does not replace a direct listener study on SoniType-generated outputs. Where models are conservative (pitch, loudness), the bias favors safety: type-checked specifications exceed the minimum perceptual clearance, so true listener discriminability may be better than the model predicts.

12.6 Threats to Validity

Construct validity. Our model-predicted metrics (d'_{model}, I_ψ) are computed from psychoacoustic models rather than direct human measurement. The model calibration study (§12.5) validates these models against published empirical data (6/6 targets passing, mean $r = 0.907$), but direct human validation with SoniType outputs is planned as future work.

External validity. Our 15 benchmark configurations span common use cases (time series, medical alarms, multivariate monitoring) but may not represent all sonification applications. In particular, musical sonification and artistic installations—where aesthetic preferences override information preservation—are outside SoniType's optimization objective.

Internal validity. The hand-tuned baselines were designed by a single auditory display researcher. Different experts might produce stronger baselines. However, the key advantage of SoniType is not that it outperforms any particular expert, but that it provides *automated, reproducible, and formally verified* optimization.

Statistical validity. Human user studies are planned but not yet conducted. Study design and power analysis are described in the appendix. Model calibration (§12.5) partially mitigates this gap by validating that the psychoacoustic models underlying our metrics match published human data (6/6 targets passing, mean $r = 0.907$).

12.7 Comparison Methodology

For the comparison in Table 4, we follow a rigorous methodology:

(1) **Equivalent specifications**: Each tool receives an equivalent task description specifying the data variables, desired mappings, and quality requirements. For tools with GUIs (Sonification Sandbox, TwoTone), settings are configured to match the SoniType specification as closely as possible.
(2) **Expert baselines**: Hand-tuned baselines are created by a researcher with 8+ years of auditory display experience, given unlimited time and access to the psychoacoustic literature.
(3) **Automated metrics**: d'_{model} and I_ψ are computed by analyzing the output audio through SoniType's psychoacoustic model pipeline, ensuring metric consistency across all tools.
(4) **Lines of code**: For declarative tools (SoniType, DSL specifications), LoC counts the specification. For imperative tools (SuperCollider, Csound), LoC counts the synthesis code plus any manual perceptual analysis code.

We emphasize that the *Verify* column in Table 4 is binary: either the tool provides compile-time formal verification of perceptual constraints (SoniType), or it does not. Partial marks indicate tools that provide some signal analysis (Faust) without formal perceptual guarantee.

12.8 Adoption Pathway

We envision a phased adoption: (1) perceptual linter for existing designs (no DSL required), (2) DSL adoption for new designs with compiler-verified guarantees, (3) integration with existing audio frameworks as a verification backend. This mirrors the adoption path of static analysis tools in software engineering.

12.9 Future Work

Several directions for future work emerge from this research:

Temporal masking. Our current masking model considers only simultaneous (spectral) masking. Forward masking (post-stimulatory suppression, ~100–200 ms) and backward masking (pre-stimulatory, ~5–10 ms) are significant for rhythmic alarm patterns. Incorporating temporal masking models from Fastl [5] would improve the linter's accuracy.

Learned perceptual models. The Zwicker/Schroeder masking model and Weber-fraction JNDs are analytically tractable but imperfect approximations. Training neural perceptual models on large-scale psychoacoustic datasets (e.g., audiometric databases) could improve prediction accuracy while preserving the differentiability needed for gradient-based optimization.

Interactive optimization. Human-in-the-loop optimization where users provide preference feedback ("these two streams sound too similar") that refines the optimizer's cost function. This would combine SoniType's formal guarantees with subjective aesthetic preferences.

Spatial audio backends. Extending the renderer to support ambisonics and binaural rendering would add a 3D spatial dimension for multi-stream separation, particularly valuable for immersive data exploration.

Cross-modal output. Extending the type system to jointly reason about visual and auditory channels would enable multimodal data display where visualization and sonification complement each other, verified against cross-modal perceptual models.

Language server protocol. Implementing an LSP backend would enable real-time perceptual linting in IDEs, with inline diagnostics showing masking violations and JND warnings as the user edits a sonification specification.

Formal verification. The current WCET analysis uses conservative cost estimation with empirical safety margins. Integrating with formal WCET tools (e.g., aiT, Bound-T) would provide provable real-time guarantees for safety-critical medical applications.

Calibration studies. Large-scale human calibration studies across diverse populations (varying age, hearing ability, musical training) would improve the accuracy of the psychoacoustic models and enable population-specific model selection.

13 Conclusion

We presented SoniType, a perceptual type system and optimizing compiler that treats sonification as a lossy coding problem over a psychoacoustically-constrained perceptual channel. The system introduces environment-dependent refinement types for psychoacoustic constraint verification, information-theoretic optimization of auditory channel capacity, and bounded-latency code generation for real-time rendering. Our Rust implementation demonstrates that the approach is both theoretically principled and practically effective.

13.1 Reproducibility

All source code, benchmark specifications, evaluation scripts, and example datasets are available in the open-source repository under the MIT/Apache-2.0 dual license. The automated evaluation pipeline can be executed with a single command (`cargo bench`), reproducing the automated quantitative results reported in this paper without requiring human subjects. Human-study protocols remain future work and no listener-study dataset is claimed here.

A Formal Typing Rules

We provide the complete set of typing rules for the SoniType perceptual type system.

T-Stream: A data source mapped to auditory parameters forms a stream.

$$\frac{\Gamma \vdash d : \text{Data}[\alpha] \quad m : \alpha \to \mathcal{P} \quad \phi_{\text{jnd}}(m(d))}{\Gamma \vdash \text{stream}(d, m) : \text{Stream}[q_m]} \tag{23}$$

T-Map: A mapping transforms data into a perceptual parameter with qualifier.

$$\frac{\Gamma \vdash d : \text{Data}[\alpha] \quad f : \alpha \to \text{Float} \quad r = [f_{\text{lo}}, f_{\text{hi}}]}{\Gamma \vdash \text{map}(d, f, r) : \text{Pitch}[q_{f,r}]} \tag{24}$$

T-Compose: Multi-stream composition checks all pairwise constraints.

$$\frac{\Gamma \vdash e_i : S_i \quad \forall i \neq j : \phi_{\text{mask}}(S_i, S_j) \wedge \phi_{\text{seg}}(S_i, S_j) \quad \phi_{\text{cog}}(\{S_i\})}{\Gamma \vdash \text{compose}(e_1, \ldots, e_k) : \text{Multi}[q_1 \sqcap \cdots \sqcap q_k]} \tag{25}$$

T-Let: Let bindings extend the typing environment.

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma, x : \tau_1 \vdash e_2 : \tau_2}{\Gamma \vdash \text{let } x = e_1 \text{ in } e_2 : \tau_2} \tag{26}$$

T-If: Conditional expressions unify branch types.

$$\frac{\Gamma \vdash e_c : \text{Bool} \quad \Gamma \vdash e_t : \tau \quad \Gamma \vdash e_f : \tau}{\Gamma \vdash \text{if } e_c \text{ then } e_t \text{ else } e_f : \tau} \tag{27}$$

B Benchmark Specifications

We list the 15 benchmark configurations used in evaluation, with stream counts and constraint parameters:

Table 11: Benchmark specification suite.

#	Name	Streams	$C_{\max}$	Domain
1	Single sinusoid	1	1	Synthetic
2	Pitch + loudness	1	1	Finance
3	Stereo time series	2	2	Climate
4	ICU 3-channel	3	3	Medical
5	Temperature 3-var	3	3	Climate
6	Stock 3-channel	3	3	Finance
7	4-channel physiology	4	4	Medical
8	5-stream complex	5	5	Synthetic
9	6-alarm palette	6	5	Medical
10	8-alarm IEC 60601	8	5	Medical
11	8-var monitoring	8	5	Industrial
12	10-stream stress	10	5	Synthetic
13	12-stream complex	12	5	Synthetic
14	14-stream extreme	14	5	Synthetic
15	16-stream maximum	16	5	Synthetic

C Masking Model Parameters

The 24 Bark critical band edge frequencies used in SoniType's masking model, following Zwicker [5]:

Table 12: Bark critical band edge frequencies (Hz).

Band	Edge (Hz)	Band	Edge (Hz)	Band	Edge (Hz)
0	20	8	920	16	3150
1	100	9	1080	17	3700
2	200	10	1270	18	4400
3	300	11	1480	19	5300
4	400	12	1720	20	6400
5	510	13	2000	21	7700
6	630	14	2320	22	9500
7	770	15	2700	23	12000
				24	15500

The Schroeder spreading function parameters used for masking computation:

- Steep (upward) slope: −25 dB/Bark
- Shallow (downward) slope: +10 dB/Bark
- Level dependence: Upward slope decreases by approximately 0.15 dB/Bark per dB of masker level above 60 dB SPL
- Temporal decay: Forward masking decays exponentially with time constant $\tau \approx 10$ ms (not modeled in current steady-state approximation)

D JND Parameter Table

Default JND thresholds used in SoniType's type checker:

Table 13: Default JND thresholds across perceptual dimensions.

Dimension	JND	Source
Pitch (< 500 Hz)	0.3% (≈ 5 cents)	Wier et al. [39]
Pitch (≥ 500 Hz)	0.6% (≈ 10 cents)	Wier et al. [39]
Loudness (broadband)	1.0 dB	Jesteadt et al. [38]
Loudness (pure tone)	0.5–1.5 dB	Jesteadt et al. [38]
Temporal onset	2 ms	Moore [29]
Spatial azimuth	1–3°	Blauert (1997)
Timbre (spectral centroid)	5%	Grey [40]

E User Study Protocol

Participants. Planned: 24 volunteers (12 sighted, 12 blind or low-vision). All must have normal or corrected-to-normal hearing confirmed by pure-tone audiometry.

Stimuli. Three 3-variable time series datasets (stock market, climate, physiological) were sonified under three conditions: (1) SoniType-optimized, (2) hand-tuned by an auditory display expert, (3) random mapping within valid parameter ranges. Presentation order was counterbalanced using a Latin square design.

Procedure. Each trial consisted of a 15-second sonification followed by three comprehension questions:

(1) **Trend identification**: "Is variable X increasing, decreasing, or stable?" (3AFC)
(2) **Outlier detection**: "At what approximate time does an anomaly occur?" (pointing task, scored within ±2 seconds)
(3) **Correlation assessment**: "Are variables X and Y positively correlated, negatively correlated, or uncorrelated?" (3AFC)

Participants completed 3 conditions × 3 datasets × 3 questions = 27 trials in approximately 45 minutes, with 5-minute breaks between blocks.

Analysis. Accuracy will be analyzed using a 3 (condition) × 2 (vision group) × 3 (question type) mixed ANOVA.

Acknowledgments

We thank the ICAD community for decades of foundational sonification research, and the reviewers for their constructive feedback.

References

[1] T. Hermann, A. Hunt, and J. G. Neuhoff. *The Sonification Handbook*. Logos Publishing House, Berlin, 2011.
[2] G. Kramer, B. Walker, T. Bonebright, P. Cook, J. Flowers, N. Miner, and J. Neuhoff. Sonification report: Status of the field and research agenda. Technical report, International Community for Auditory Display, 1999.
[3] B. N. Walker and M. A. Nees. Theory of sonification. In *The Sonification Handbook*, pp. 9–39. Logos, 2011.
[4] A. S. Bregman. *Auditory Scene Analysis: The Perceptual Organization of Sound*. MIT Press, 1990.
[5] E. Zwicker and H. Fastl. *Psychoacoustics: Facts and Models*. Springer, 3rd edition, 2013.
[6] ECRI Institute. Top 10 health technology hazards for 2014. *Health Devices*, 42(11), 2013.
[7] The Joint Commission. National Patient Safety Goal on alarm management. Sentinel Event Alert 50, 2013.
[8] International Electrotechnical Commission. IEC 60601-1-8: Medical electrical equipment—Part 1-8: General requirements for basic safety and essential performance—Collateral standard: General requirements, tests and guidance for alarm systems. 2012.
[9] B. N. Walker and J. T. Cothran. Sonification Sandbox: A graphical toolkit for auditory graphs. In *Proc. ICAD*, pp. 161–163, 2003.
[10] Monash University. TwoTone: Turn your data into music. https://twotone.io, 2020.
[11] Highsoft AS. Highcharts Sonification Module. https://www.highcharts.com/docs/sonification, 2022.
[12] J. McCartney. Rethinking the computer music language: SuperCollider. *Computer Music Journal*, 26(4):61–68, 2002.
[13] Y. Orlarey, D. Fober, and S. Letz. FAUST: An efficient functional approach to DSP programming. In *New Computational Paradigms for Computer Music*, pp. 65–96. IRCAM, 2009.
[14] R. Boulanger, editor. *The Csound Book*. MIT Press, 2000.
[15] G. Wang and P. R. Cook. ChucK: A concurrent, on-the-fly audio programming language. In *Proc. ICMC*, 2003.
[16] T. Painter and A. Spanias. Perceptual coding of digital audio. *Proc. IEEE*, 88(4):451–515, 2000.
[17] M. Bosi and R. E. Goldberg. *Introduction to Digital Audio Coding and Standards*. Springer, 2002.
[18] M. R. Schroeder, B. S. Atal, and J. L. Hall. Optimizing digital speech coders by exploiting masking properties of the human ear. *J. Acoust. Soc. Am.*, 66(6):1647–1652, 1979.
[19] B. R. Glasberg and B. C. J. Moore. A model of loudness applicable to time-varying sounds. *J. Audio Eng. Soc.*, 50(5):331–342, 2002.
[20] T. Freeman and F. Pfenning. Refinement types for ML. In *Proc. PLDI*, pp. 268–277, 1991.
[21] P. M. Rondon, M. Kawaguchi, and R. Jhala. Liquid types. In *Proc. PLDI*, pp. 159–169, 2008.
[22] R. Milner. A theory of type polymorphism in programming. *J. Computer and System Sciences*, 17(3):348–375, 1978.
[23] L. Damas and R. Milner. Principal type-schemes for functional programs. In *Proc. POPL*, pp. 207–212, 1982.
[24] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proc. PLDI*, pp. 519–530, 2013.
[25] M. Püschel, J. M. F. Moura, J. R. Johnson, D. Padua, M. M. Veloso, B. W. Singer, J. Xiong, F. Franchetti, A. Gacic, Y. Voronenko, K. Chen, R. W. Johnson, and N. Rizzolo. SPIRAL: Code generation for DSP transforms. *Proc. IEEE*, 93(2):232–275, 2005.
[26] World Wide Web Consortium. Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 2023.
[27] European Parliament. Directive (EU) 2019/882 on the accessibility requirements for products and services (European Accessibility Act). *Official Journal of the European Union*, 2019.
[28] J. H. Flowers, D. C. Buhman, and K. D. Turnage. Cross-modal equivalence of visual and auditory scatterplots for exploring bivariate data samples. *Human Factors*, 47(1):65–77, 2005.
[29] B. C. J. Moore. *An Introduction to the Psychology of Hearing*. Emerald, 6th edition, 2012.
[30] N. Cowan. The magical number 4 in short-term memory: A reconsideration of mental storage capacity. *Behavioral and Brain Sciences*, 24(1):87–114, 2001.
[31] R. Wilhelm, J. Engblom, A. Ermedahl, N. Holsti, S. Thesing, D. Whalley, G. Bernat, C. Ferdinand, R. Heckmann, T. Mitra, F. Mueller, I. Puaut, P. Puschner, J. Staschulat, and P. Stenström. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Trans. Embedded Computing Systems*, 7(3):36:1–36:53, 2008.
[32] International Telecommunication Union. ITU-R BS.1534-3: Method for the subjective assessment of intermediate quality level of audio systems (MUSHRA). 2014.
[33] World Wide Web Consortium. Web Audio API. W3C Recommendation, 2021.
[34] D. M. Green and J. A. Swets. *Signal Detection Theory and Psychophysics*. Wiley, 1966.
[35] L. P. A. S. Van Noorden. Temporal coherence in the perception of tone sequences. PhD thesis, Eindhoven University of Technology, 1975.
[36] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions—I. *Mathematical Programming*, 14(1):265–294, 1978.
[37] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
[38] W. Jesteadt, C. C. Wier, and D. M. Green. Intensity discrimination as a function of frequency and sensation level. *J. Acoust. Soc. Am.*, 61(1):169–177, 1977.
[39] C. C. Wier, W. Jesteadt, and D. M. Green. Frequency discrimination as a function of frequency and sensation level. *J. Acoust. Soc. Am.*, 61(1):178–184, 1977.
[40] J. M. Grey. Multidimensional perceptual scaling of musical timbres. *J. Acoust. Soc. Am.*, 61(5):1270–1277, 1977.
[41] M. Cvach. Monitor alarm fatigue: An integrative review. *Biomedical Instrumentation & Technology*, 46(4):268–277, 2012.
[42] G. Keidser, H. Dillon, M. Flax, T. Ching, and S. Brewer. The NAL-NL2 prescription procedure. *Audiology Research*, 1(1):e24, 2011.
[43] R. Plomp and W. J. M. Levelt. Tonal consonance and critical bandwidth. *J. Acoust. Soc. Am.*, 38(4):548–560, 1965.
[44] B. R. Glasberg and B. C. J. Moore. Derivation of auditory filter shapes from notched-noise data. *Hearing Research*, 47(1-2):103–138, 1990.
[45] H. Traunmüller. Analytical expressions for the tonotopic sensory scale. *J. Acoust. Soc. Am.*, 88(1):97–100, 1990.
[46] S. McAdams, S. Winsberg, S. Donnadieu, G. De Soete, and J. Krimphoff. Perceptual scaling of synthesized musical timbres: Common dimensions, specificities, and latent subject classes. *Psychological Research*, 58(3):177–192, 1995.
[47] M. Puckette. Pure Data: Another integrated computer music environment. In *Proc. International Computer Music Conference*, pp. 37–41, 1996.
[48] IRIS Data Management Center. Data services products: Waveforms, metadata, and derived products. https://ds.iris.edu, 2012.
[49] M. Tangermann, K.-R. Müller, A. Aertsen, N. Birbaumer, C. Braun, C. Brunner, R. Leeb, C. Mehring, K. J. Miller, G. R. Müller-Putz, G. Nolte, G. Pfurtscheller, H. Preissl, G. Schalk, A. Schlögl, C. Vidaurre, S. Waldert, and B. Blankertz. Review of the BCI Competition IV. *Frontiers in Neuroscience*, 6:55, 2012.
[50] E. Dong, H. Du, and L. Gardner. An interactive web-based dashboard to track COVID-19 in real time. *The Lancet Infectious Diseases*, 20(5):533–534, 2020.
[51] G. T. Fechner. *Elemente der Psychophysik*. Breitkopf und Härtel, Leipzig, 1860.
[52] S. G. Hart and L. E. Staveland. Development of NASA-TLX (Task Load Index): Results of empirical and theoretical research. In P. A. Hancock and N. Meshkati, editors, *Human Mental Workload*, pp. 139–183. North-Holland, 1988.
[53] W. S. Cleveland and R. McGill. Graphical perception: Theory, experimentation, and application to the development of graphical methods. *Journal of the American Statistical Association*, 79(387):531–554, 1984.
[54] J. Heer and M. Bostock. Crowdsourcing graphical perception: Using mechanical turk to assess visualization design. In *Proceedings of CHI 2010*, pp. 203–212. ACM, 2010.
[55] G. A. Gescheider. *Psychophysics: The Fundamentals*. Lawrence Erlbaum Associates, 3rd edition, 1997.

[56] R. T. Verrillo. Psychophysics of vibrotactile stimulation. *Journal of the Acoustical Society of America*, 77(1):225–232, 1985.
[57] M. O. Ernst and M. S. Banks. Humans integrate visual and haptic information in a statistically optimal fashion. *Nature*, 415(6870):429–433, 2002.
[58] C. D. Wickens. Multiple resources and mental workload. *Human Factors*, 50(3):449–455, 2008.
[59] E. Zwicker and E. Terhardt. Analytical expressions for critical-band rate and critical bandwidth as a function of frequency. *J. Acoust. Soc. Am.*, 68(5):1523–1525, 1980.
[60] A. Friberg and J. Sundberg. Time discrimination in a monotonic, isochronous sequence. *J. Acoust. Soc. Am.*, 98(5):2524–2535, 1995.